

Project Documentation

Table of Contents

Requirement Specification Document:	2
Functional Requirements	3
FR1: User Management	3
FR2: Event Management	3
FR3: Registration System	3
FR4: Session Management	3
FR5: Basic Venue Management	4
FR6: Search & Discovery	4
FR7: Check-in System	4
FR8: Basic Interaction Features	4
FR09: Payment Processing	4
Detailed Design Document:	6
Why This System Was Needed	10
Implementation and Testing:	10
Implementation of code	10
A. Code implementation project file design:	11
B. Package 1	12
1) Main.java	12
C. Package 2	14
2) Event.java	14
3) PaymentStatus.java	16
4) Registration.java	16
5) Session.java	18
6) TicketType.java	20
7) User.java	20
8) Venue.java	22
D. Package 3	23
9) EventService.java	23
10) RegistrationService.java	24
11) SessionService.java	26
12) UserService.java	27
Output	28

Testing classes	29
1.EventServiceTest.java	29
1.Test Unit 1 Result:.....	30
2.RegistrationServiceTest.java	30
2.Test unit 2 Result.....	32
3.SessionServiceTest.java	32
3.Test unit 3 Result:.....	33
4.UserServiceTest.java	34
4.Test unit 4 Result.....	35
All units test Results:	35
Bug report table:	37
Issues Encountered During Implementation and Testing.....	37

Requirement Specification Document:

System Name

Smart Event Management & Networking Platform

System Overview

This system is designed to help event organizers create, manage, and monitor events efficiently. It improves attendee engagement, simplifies registration, and enables better session and venue management.

The platform supports different user roles and provides tools for event scheduling, registration, and interaction.

User Roles

- **Admin**
- **Organizer**
- **Staff**
- **Attendee**
- **Payment Gateway**

Functional Requirements

FR1: User Management

- Users can register and log in.
- Role-based access control is implemented.
- Super Admin can manage all users.

FR2: Event Management

- Organizers can:
 - I. Create events
 - II. Edit/delete events
 - III. Duplicate events (template)
- Dashboard shows:
 - I. Total registrations
 - II. Attendance stats

FR3: Registration System

- Users can register for events
- Different ticket types (Standard, VIP)
- Generate QR code for confirmation
- Export attendee list (CSV or pdf)

FR4: Session Managemnt

- Organizers can:
 - I. Create sessions
 - II. Assign speakers
 - III. Set time & venue
- Detect scheduling conflicts
- Show session capacity (Available / Full)

FR5: Basic Venue Management

- Define rooms/halls
- Assign sessions to rooms
- Set capacity per room

FR6: Search & Discovery

- Search for:
 - I. Sessions
 - II. Attendees
- Filter by:
 - I. Time
 - II. Topic
 - III. Speaker

FR7: Check-in System

- QR code check-in
- Staff can mark attendance

FR8: Basic Interaction Features

- Q&A submission
- Session notes

FR09: Payment Processing

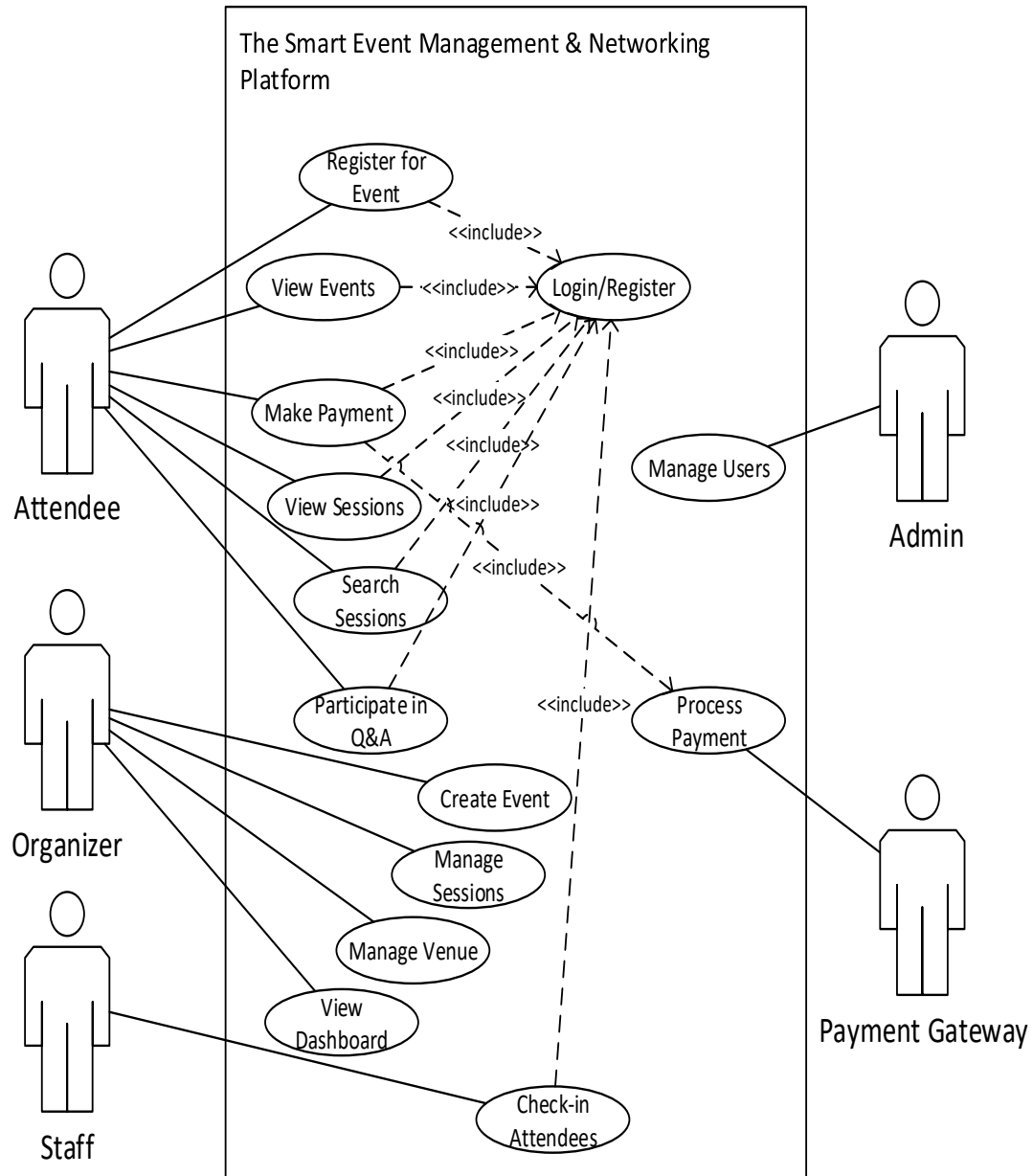
The system shall allow attendees to securely make payments for event registration through an integrated external payment gateway. Upon successful payment, the system shall confirm the registration and generate a QR code for the attendee. In case of payment failure, the system shall display an appropriate error message and prevent registration confirmation.

❖ **Non-Functional Requirements (NFRs)**

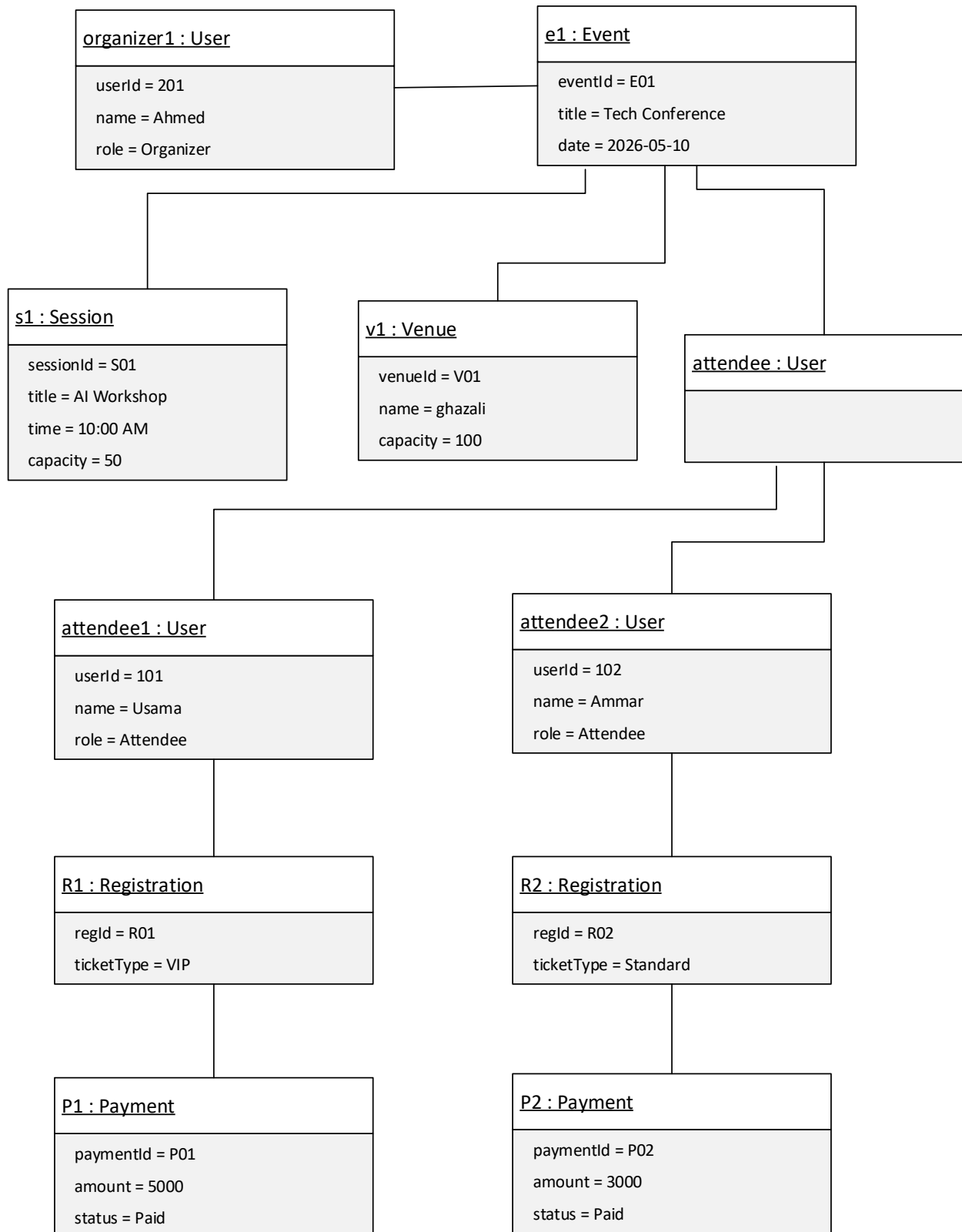
- **NFR-1: Modularity (Architecture):** The system must be designed using Object-Oriented principles (encapsulation, inheritance, polymorphism) to ensure high re-usability for future extensions.
- **NFR-2: Performance:** Conflict detection algorithms should execute in under 2 seconds, even with 100+ scheduled sessions.
- **NFR-3: Reliability:** Data must be persistently stored (either via a relational database like MySQL using JDBC, and MongoDB).
- **NFR-4: Security:** Passwords must be securely hashed, and strict access control checks must occur before any data modification.

Detailed Design Document:

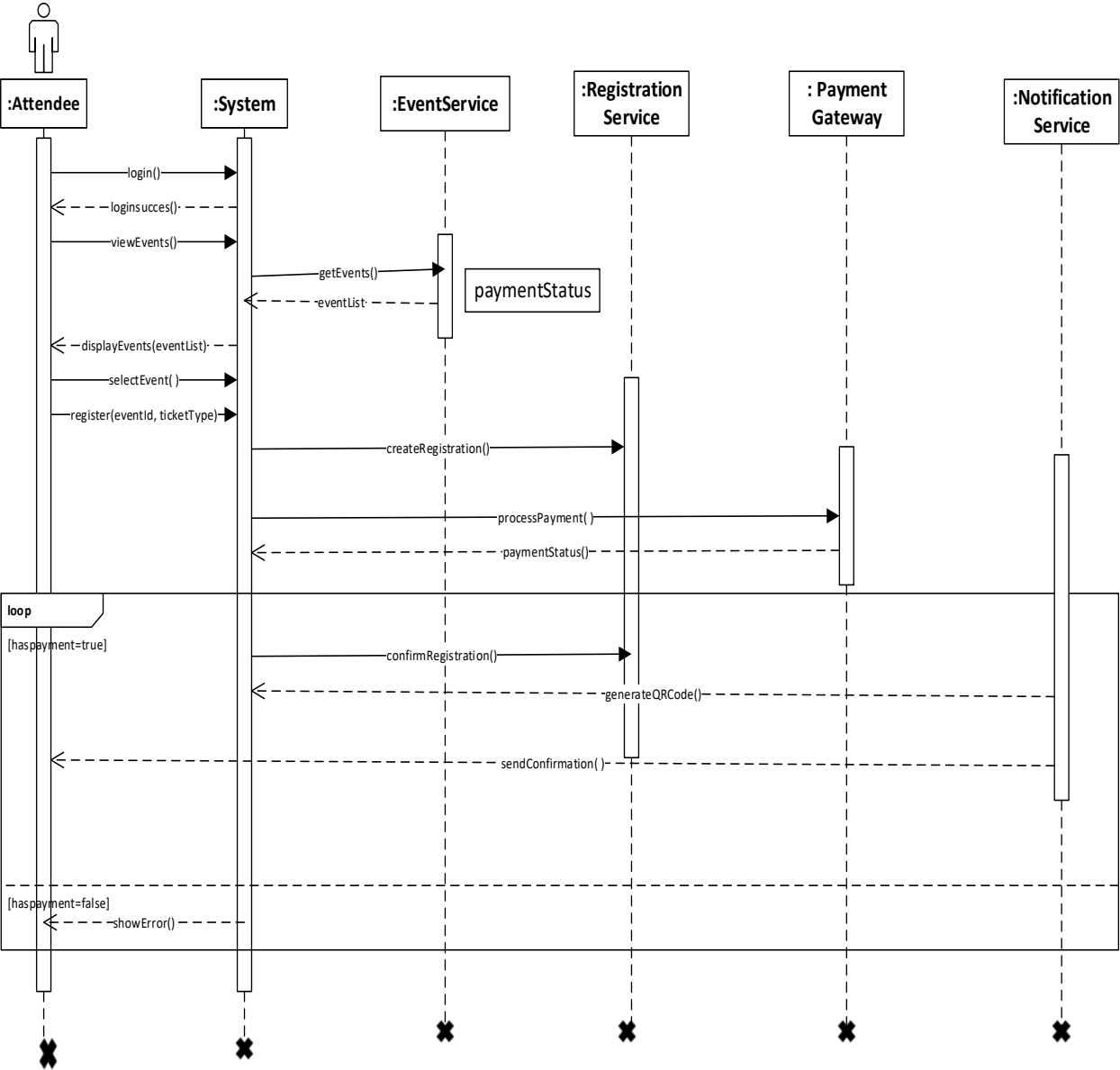
USE CASE DIAGRAM



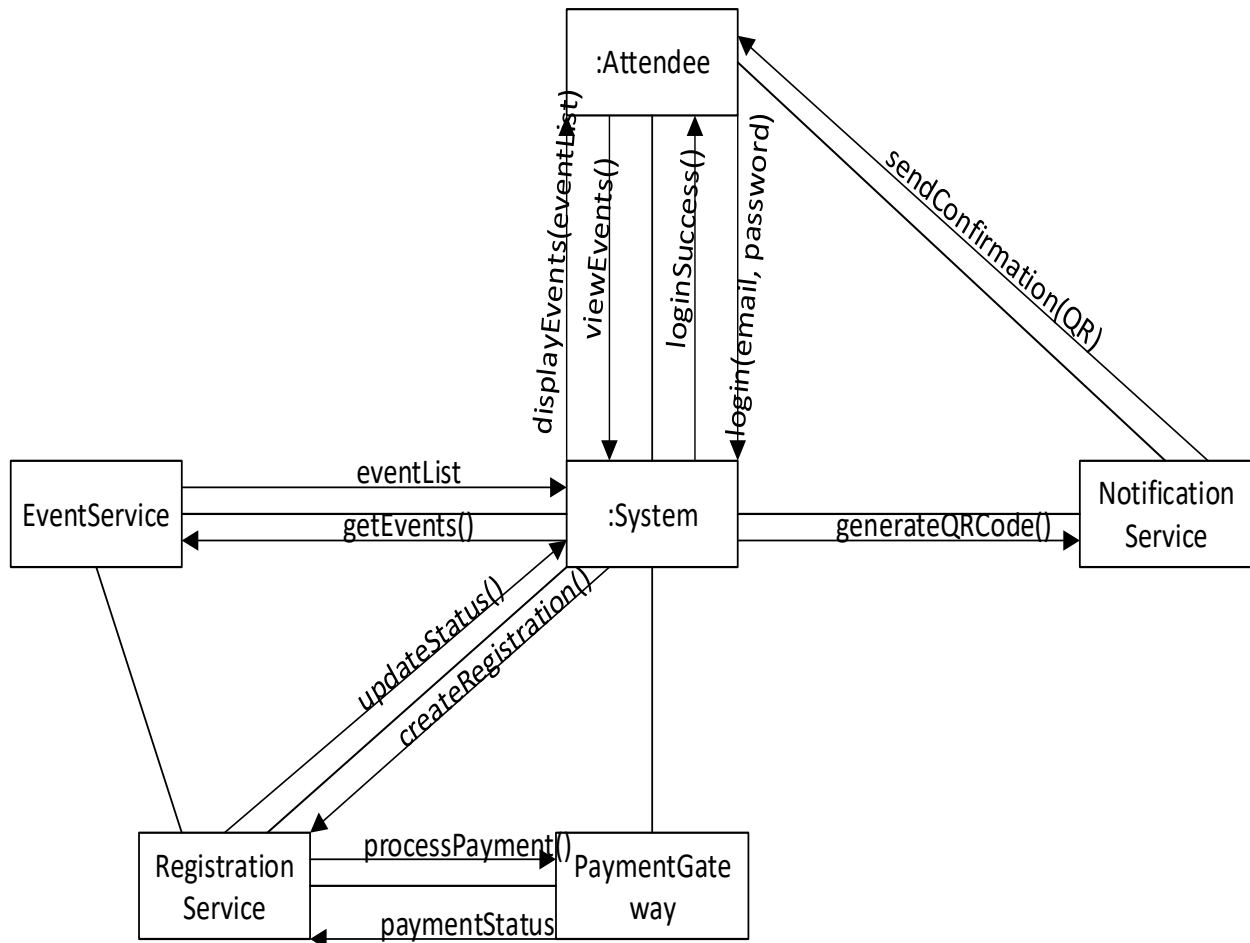
Object diagram



Sequence diagram



Communication diagram



Why This System Was Needed

In this project, we solved the problem of manual event management. Normally, in events like seminars, workshops, or conferences, organizers have to manage users, event details, registrations, sessions, payments, and notifications manually. This can cause many issues like duplicate registration, payment mistakes, missing records, and poor communication.

Our system provides one centralized platform where events can be created, attendees can register, payments can be verified, and notifications can be sent after successful registration. This makes the process more organized, faster, and reliable.

So, the main problem we solved is that we replaced the manual and confusing event management process with a proper digital system.

Implementation and Testing:

- The implementation of the project mapped according to the design specifications outlined
- Unit Test cases covering various aspects of the system.
- Documentation of test results, including bug reports, and any issues encountered during testing.
- Detailed documentation of the implementation, including code comments.

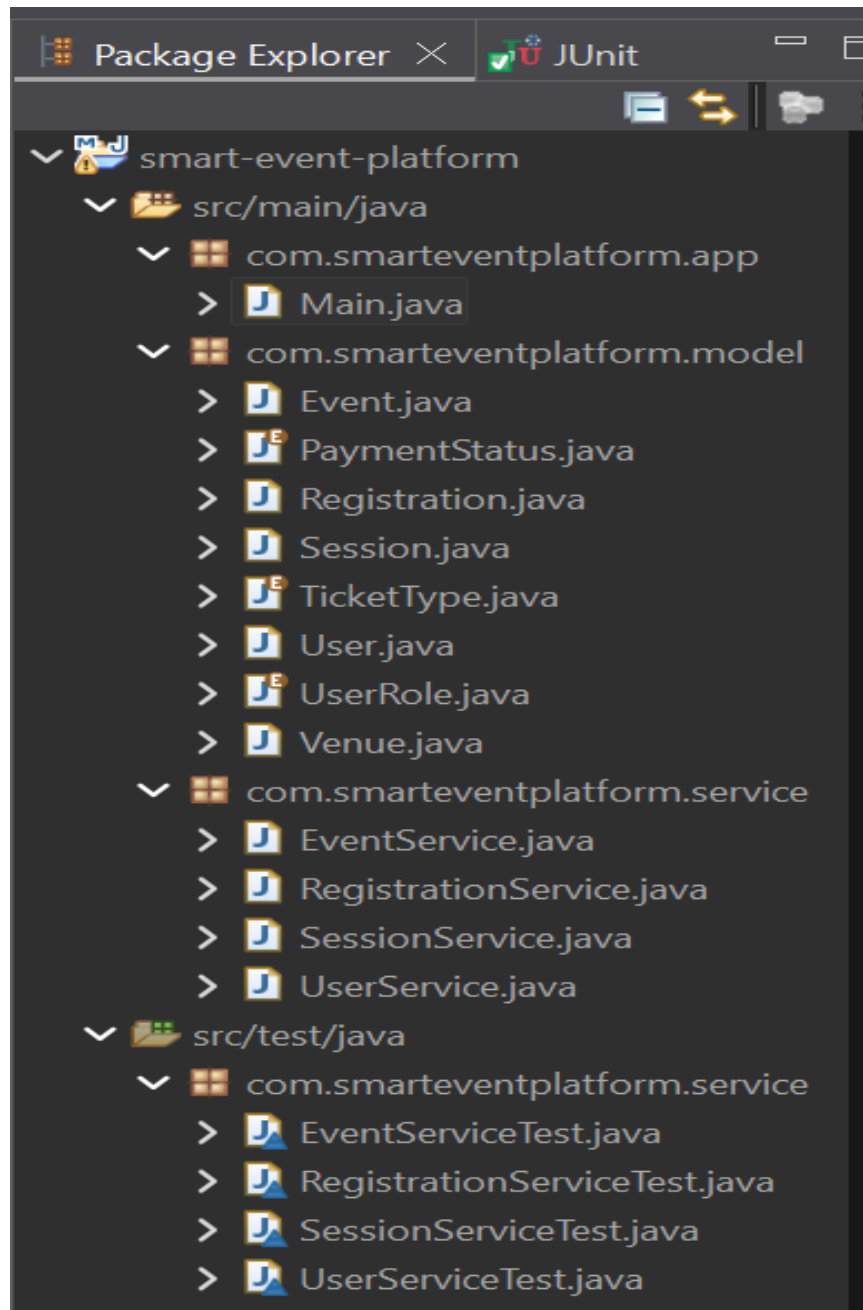
Implementation of code

The project was implemented according to the system design using Java object-oriented programming concepts. Separate classes were created for users, events, sessions, venues, and registrations. Service classes were used to manage system operations like event creation, login, registration, and payment processing.

Clean code principles and refactoring were applied to improve readability and maintainability. Unnecessary comments and repeated code were removed, method names were simplified, and the code structure was organized clearly.

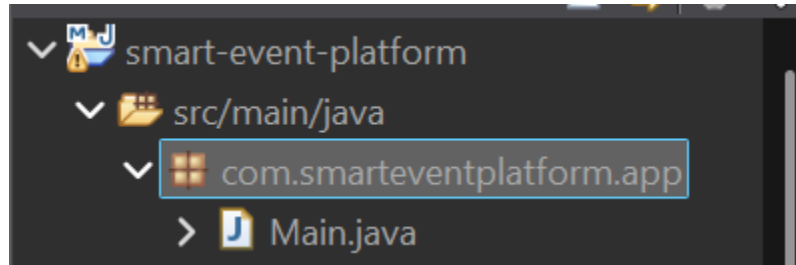
Error handling was also implemented using exception handling (try-catch) to manage invalid inputs, duplicate registrations, login errors, session conflicts, and password validation problems.

A. Code implementation project file design:



B. Package 1

1) Main.java



```
package
com.smarteventplatform.app;
```

```
import com.smarteventplatform.model.Event;
import com.smarteventplatform.model.PaymentStatus;
import com.smarteventplatform.model.Registration;
import com.smarteventplatform.model.Session;
import com.smarteventplatform.model.TicketType;
import com.smarteventplatform.model.User;
import com.smarteventplatform.model.UserRole;
import com.smarteventplatform.model.Venue;
import com.smarteventplatform.service.EventService;
import com.smarteventplatform.service.RegistrationService;
import com.smarteventplatform.service.SessionService;
import com.smarteventplatform.service.UserService;

public class Main {
    public static void main(String[] args) {
        UserService userService = new UserService();
        EventService eventService = new EventService();
        SessionService sessionService = new SessionService();
        RegistrationService registrationService = new RegistrationService();

        System.out.println("Welcome to our  \n");
        System.out.println("Smart Event Management & Networking Platform\n\n");

        try {
            User admin = userService.registerUser("Admin khan", "admin@events.com",
"admin123", UserRole.ADMIN);
            User organizer = userService.registerUser("Ammar Kabeer",
"ammar@events.com", "pass1234", UserRole.ORGANIZER);
            User attendeeOne = userService.registerUser("Usama", "usama@gmail.com",
"pass1234", UserRole.ATTENDEE);
            User attendeeTwo = userService.registerUser("Ammar", "ammar@gmail.com",
"pass1234", UserRole.ATTENDEE);

            printSection("Registered Users");
            userService.getAllUsers().forEach(System.out::println);

            User loggedInUser = userService.login("ammar@events.com", "pass1234");
            System.out.println("\nLogged in as: " + loggedInUser.getName());

            Venue venue = new Venue("V01", "Ghazkhan Hall", 100);
            Event event = eventService.createEvent("Tech Conference 2026", "2026-05-
10", organizer);

            Session firstSession = sessionService.createSession("AI Workshop", "Dr.
Sara", "10:00 AM", venue, 50);
```

```

        Session secondSession = sessionService.createSession("Cloud Computing",
"Mr. Bilal", "02:00 PM", venue, 40);

        event.addSession(firstSession);
        event.addSession(secondSession);

        printSection("Event Details");
        System.out.println(event);
        event.getSessions().forEach(System.out::println);

        Registration firstRegistration =
registrationService.register(attendeeOne, event, TicketType.VIP);
        Registration secondRegistration =
registrationService.register(attendeeTwo, event, TicketType.STANDARD);

        registrationService.processPayment(firstRegistration, true);
        registrationService.processPayment(secondRegistration, false);

        printSection("Registrations");
        registrationService.getAllRegistrations().forEach(System.out::println);

        eventService.updateEvent(event.getEventId(), "Tech Conference 2026
Updated", null, admin);

        printSection("Updated Event");
        System.out.println(eventService.findById(event.getEventId()));

        printSection("Payment Summary");
        System.out.println("Paid registrations: " +
countPaidRegistrations(registrationService));

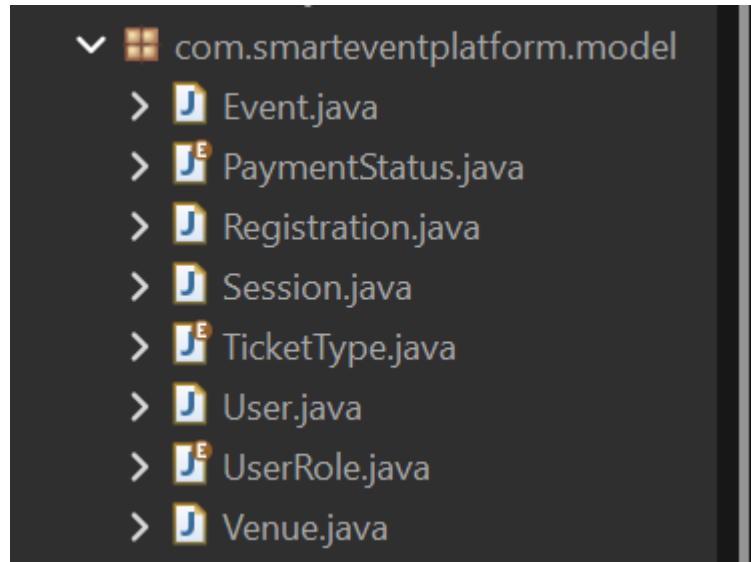
    } catch (RuntimeException exception) {
        System.err.println("Error: " + exception.getMessage());
    } finally {
        printSection("Application Status");
        System.out.println("Application execution completed.");
        System.out.println("Thank you for using Smart Event Management &
Networking Platform.");
    }
}

private static long countPaidRegistrations(RegistrationService
registrationService) {
    return registrationService.getAllRegistrations().stream()
        .filter(registration -> registration.getPaymentStatus() ==
PaymentStatus.PAID)
        .count();
}

private static void printSection(String title) {
    System.out.println("\n--- " + title + " ---");
}
}

```

C. Package 2



2) Event.java

```
package com.smarteventplatform.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Event {
    private final String eventId;
    private String title;
    private String date;
    private final User organizer;
    private final List<Session> sessions = new ArrayList<>();

    public Event(String eventId, String title, String date, User organizer) {
        validate(eventId, title, date, organizer);
        this.eventId = eventId.trim();
        this.title = title.trim();
        this.date = date.trim();
        this.organizer = organizer;
    }

    private static void validate(String eventId, String title, String date, User
organizer) {
        if (isBlank(eventId)) {
            throw new IllegalArgumentException("Event ID cannot be empty.");
        }
        if (isBlank(title)) {
            throw new IllegalArgumentException("Event title cannot be empty.");
        }
    }
}
```

```

    }
    if (isBlank(date)) {
        throw new IllegalArgumentException("Event date cannot be empty.");
    }
    if (organizer == null) {
        throw new IllegalArgumentException("Organizer is required.");
    }
    if (organizer.getRole() != UserRole.ORGANIZER && organizer.getRole() !=
UserRole.ADMIN) {
        throw new IllegalArgumentException("Only an Organizer or Admin can create
events.");
    }
}

public void addSession(Session session) {
    if (session == null) {
        throw new IllegalArgumentException("Session is required.");
    }
    if (hasScheduleConflict(session)) {
        throw new IllegalStateException(
            "Scheduling conflict: Venue \"" + session.getVenue().getName()
            + "\" is already booked at " + session.getTimeSlot()
        );
    }
    sessions.add(session);
}

private boolean hasScheduleConflict(Session newSession) {
    return sessions.stream().anyMatch(existingSession ->
existingSession.getVenue().getVenueId().equalsIgnoreCase(newSession.getVenue().getVen
ueId())
        &&
existingSession.getTimeSlot().equalsIgnoreCase(newSession.getTimeSlot())
    );
}

private static boolean isBlank(String value) {
    return value == null || value.isBlank();
}

public String getEventId() {
    return eventId;
}

public String getTitle() {
    return title;
}

public String getDate() {
    return date;
}

public User getOrganizer() {
    return organizer;
}

```

```

    }

    public List<Session> getSessions() {
        return Collections.unmodifiableList(sessions);
    }

    public void setTitle(String title) {
        if (!isBlank(title)) {
            this.title = title.trim();
        }
    }

    public void setDate(String date) {
        if (!isBlank(date)) {
            this.date = date.trim();
        }
    }

    @Override
    public String toString() {
        return String.format(
            "[Event %s] \"%s\" on %s | Organizer: %s | Sessions: %d",
            eventId,
            title,
            date,
            organizer.getName(),
            sessions.size()
        );
    }
}

```

3) PaymentStatus.java

```

package com.smarteventplatform.model;

public enum PaymentStatus {
    PAID,
    PENDING,
    FAILED
}

```

4) Registration.java

```

package com.smarteventplatform.model;

public class Registration {
    private final String registrationId;
    private final User attendee;
    private final Event event;
}

```



```

private final TicketType ticketType;
private PaymentStatus paymentStatus;
private String qrCode;

public Registration(String registrationId, User attendee, Event event, TicketType
ticketType) {
    validate(registrationId, attendee, event, ticketType);
    this.registrationId = registrationId.trim();
    this.attendee = attendee;
    this.event = event;
    this.ticketType = ticketType;
    this.paymentStatus = PaymentStatus.PENDING;
}

private static void validate(String registrationId, User attendee, Event event,
TicketType ticketType) {
    if (isBlank(registrationId)) {
        throw new IllegalArgumentException("Registration ID cannot be empty.");
    }
    if (attendee == null) {
        throw new IllegalArgumentException("Attendee is required.");
    }
    if (event == null) {
        throw new IllegalArgumentException("Event is required.");
    }
    if (ticketType == null) {
        throw new IllegalArgumentException("Ticket type is required.");
    }
    if (attendee.getRole() != UserRole.ATTENDEE) {
        throw new IllegalArgumentException("Only Attendees can register for
events.");
    }
}

public void confirmPayment() {
    paymentStatus = PaymentStatus.PAID;
    qrCode = generateQrCode();
}

public void failPayment() {
    paymentStatus = PaymentStatus.FAILED;
    qrCode = null;
}

private String generateQrCode() {
    return "QR-" + registrationId + "-" + event.getEventId();
}

private static boolean isBlank(String value) {
    return value == null || value.isBlank();
}

public String getRegistrationId() {
    return registrationId;
}

```

```

    public User getAttendee() {
        return attendee;
    }

    public Event getEvent() {
        return event;
    }

    public TicketType getTicketType() {
        return ticketType;
    }

    public PaymentStatus getPaymentStatus() {
        return paymentStatus;
    }

    public String getQrCode() {
        return qrCode;
    }

    @Override
    public String toString() {
        String displayQrCode = qrCode == null ? "N/A" : qrCode;
        return String.format(
            "[Reg %s] %s -> \"%s\" | Ticket: %s | Status: %s | QR: %s",
            registrationId,
            attendee.getName(),
            event.getTitle(),
            ticketType,
            paymentStatus,
            displayQrCode
        );
    }
}

```

5) Session.java

```

package com.smarteventplatform.model;

public class Session {
    private final String sessionId;
    private final String title;
    private final String speaker;
    private final String timeSlot;
    private final Venue venue;
    private final int capacity;

    public Session(String sessionId, String title, String speaker, String timeSlot,
        Venue venue, int capacity) {
        validate(sessionId, title, speaker, timeSlot, venue, capacity);
        this.sessionId = sessionId.trim();
    }
}

```

```

        this.title = title.trim();
        this.speaker = speaker.trim();
        this.timeSlot = timeSlot.trim();
        this.venue = venue;
        this.capacity = capacity;
    }

    private static void validate(String sessionId, String title, String speaker,
String timeSlot, Venue venue, int capacity) {
        if (isBlank(sessionId)) {
            throw new IllegalArgumentException("Session ID cannot be empty.");
        }
        if (isBlank(title)) {
            throw new IllegalArgumentException("Session title cannot be empty.");
        }
        if (isBlank(speaker)) {
            throw new IllegalArgumentException("Speaker name cannot be empty.");
        }
        if (isBlank(timeSlot)) {
            throw new IllegalArgumentException("Time slot cannot be empty.");
        }
        if (venue == null) {
            throw new IllegalArgumentException("Venue is required.");
        }
        if (capacity <= 0) {
            throw new IllegalArgumentException("Session capacity must be positive.");
        }
        if (capacity > venue.getCapacity()) {
            throw new IllegalArgumentException("Session capacity cannot exceed venue
capacity.");
        }
    }

    private static boolean isBlank(String value) {
        return value == null || value.isBlank();
    }

    public String getSessionId() {
        return sessionId;
    }

    public String getTitle() {
        return title;
    }

    public String getSpeaker() {
        return speaker;
    }

    public String getTimeSlot() {
        return timeSlot;
    }

    public Venue getVenue() {
        return venue;
    }

```

```

    }

    public int getCapacity() {
        return capacity;
    }

    @Override
    public String toString() {
        return String.format(
            "[Session %s] \"%s\" by %s @ %s | Venue: %s | Capacity: %d",
            sessionId,
            title,
            speaker,
            timeSlot,
            venue.getName(),
            capacity
        );
    }
}

```

6) TicketType.java

```

package com.smarteventplatform.model;

public enum TicketType {
    STANDARD,
    VIP
}

```

7) User.java

```

package com.smarteventplatform.model;

import java.util.Objects;

public class User {
    private final int userId;
    private final String name;
    private final String email;
    private final String password;
    private final UserRole role;

    public User(int userId, String name, String email, String password, UserRole
role) {
        validate(name, email, password, role);
        this.userId = userId;
        this.name = name.trim();
        this.email = email.trim().toLowerCase();
        this.password = password;
        this.role = role;
    }

    private static void validate(String name, String email, String password, UserRole
role) {

```

```

        if (isBlank(name)) {
            throw new IllegalArgumentException("Name cannot be empty.");
        }
        if (isBlank(email)) {
            throw new IllegalArgumentException("Email cannot be empty.");
        }
        if (password == null || password.length() < 6) {
            throw new IllegalArgumentException("Password must be at least 6
characters.");
        }
        if (role == null) {
            throw new IllegalArgumentException("User role is required.");
        }
    }

    private static boolean isBlank(String value) {
        return value == null || value.isBlank();
    }

    public int getUserId() {
        return userId;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getPassword() {
        return password;
    }

    public UserRole getRole() {
        return role;
    }

    @Override
    public boolean equals(Object object) {
        if (this == object) {
            return true;
        }
        if (!(object instanceof User user)) {
            return false;
        }
        return userId == user.userId;
    }

    @Override
    public int hashCode() {
        return Objects.hash(userId);
    }

```

```

@Override
public String toString() {
    return String.format("[User #%d] %s | %s | Role: %s", userId, name, email,
role);
}
}

```

8) Venue.java

```

package com.smarteventplatform.model;

public class Venue {
    private final String venueId;
    private final String name;
    private final int capacity;

    public Venue(String venueId, String name, int capacity) {
        if (isBlank(venueId)) {
            throw new IllegalArgumentException("Venue ID cannot be empty.");
        }
        if (isBlank(name)) {
            throw new IllegalArgumentException("Venue name cannot be empty.");
        }
        if (capacity <= 0) {
            throw new IllegalArgumentException("Venue capacity must be positive.");
        }

        this.venueId = venueId.trim();
        this.name = name.trim();
        this.capacity = capacity;
    }

    private static boolean isBlank(String value) {
        return value == null || value.isBlank();
    }

    public String getVenueId() {
        return venueId;
    }

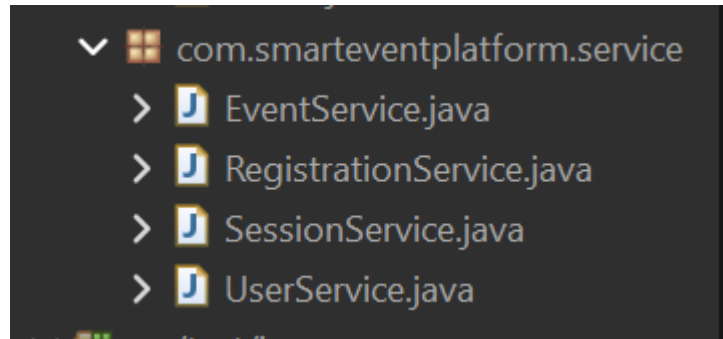
    public String getName() {
        return name;
    }

    public int getCapacity() {
        return capacity;
    }

    @Override
    public String toString() {
        return String.format("[Venue %s] %s | Capacity: %d", venueId, name,
capacity);
    }
}

```

D. Package 3



9) EventService.java

```
package com.smarteventplatform.service;

import com.smarteventplatform.model.Event;
import com.smarteventplatform.model.User;
import com.smarteventplatform.model.UserRole;

import java.util.ArrayList;
import java.util.List;

public class EventService {
    private final List<Event> events = new ArrayList<>();
    private int nextId = 1;

    public Event createEvent(String title, String date, User organizer) {
        String eventId = String.format("E%03d", nextId++);
        Event event = new Event(eventId, title, date, organizer);
        events.add(event);
        return event;
    }

    public void updateEvent(String eventId, String newTitle, String newDate, User requester) {
        Event event = findById(eventId);
        checkEventPermission(event, requester);
        event.setTitle(newTitle);
        event.setDate(newDate);
    }

    public void deleteEvent(String eventId, User requester) {
        Event event = findById(eventId);
        checkEventPermission(event, requester);
        events.remove(event);
    }

    public List<Event> getAllEvents() {
        return new ArrayList<>(events);
    }
}
```

```

    public Event findById(String eventId) {
        return events.stream()
            .filter(event -> event.getEventId().equalsIgnoreCase(eventId))
            .findFirst()
            .orElseThrow(() -> new IllegalArgumentException("Event not found: " +
eventId));
    }

    public List<Event> searchByTitle(String keyword) {
        if (keyword == null || keyword.isBlank()) {
            return List.of();
        }

        String normalizedKeyword = keyword.toLowerCase();
        return events.stream()
            .filter(event ->
event.getTitle().toLowerCase().contains(normalizedKeyword))
            .toList();
    }

    private void checkEventPermission(Event event, User requester) {
        if (requester == null) {
            throw new SecurityException("Permission denied.");
        }

        boolean isOwner = event.getOrganizer().equals(requester);
        boolean isAdmin = requester.getRole() == UserRole.ADMIN;

        if (!isOwner && !isAdmin) {
            throw new SecurityException("Permission denied: not the event
organizer.");
        }
    }
}

```

10) RegistrationService.java

```

package com.smarteventplatform.service;

import com.smarteventplatform.model.Event;
import com.smarteventplatform.model.Registration;
import com.smarteventplatform.model.TicketType;
import com.smarteventplatform.model.User;

import java.util.ArrayList;
import java.util.List;

public class RegistrationService {
    private final List<Registration> registrations = new ArrayList<>();
    private int nextId = 1;
}

```



```

public Registration register(User attendee, Event event, TicketType ticketType) {
    if (attendee == null) {
        throw new IllegalArgumentException("Attendee is required.");
    }
    if (event == null) {
        throw new IllegalArgumentException("Event is required.");
    }
    if (isAlreadyRegistered(attendee, event)) {
        throw new IllegalStateException("Attendee already registered for this
event.");
    }

    String registrationId = String.format("R%03d", nextId++);
    Registration registration = new Registration(registrationId, attendee, event,
ticketType);
    registrations.add(registration);
    return registration;
}

public void processPayment(Registration registration, boolean paymentSuccess) {
    if (registration == null) {
        throw new IllegalArgumentException("Registration is required.");
    }

    if (paymentSuccess) {
        registration.confirmPayment();
        return;
    }
    registration.failPayment();
}

public List<Registration> getRegistrationsByEvent(String eventId) {
    return registrations.stream()
        .filter(registration ->
registration.getEvent().getEventId().equalsIgnoreCase(eventId))
        .toList();
}

public List<Registration> getAllRegistrations() {
    return new ArrayList<>(registrations);
}

public Registration findById(String registrationId) {
    return registrations.stream()
        .filter(registration ->
registration.getRegistrationId().equalsIgnoreCase(registrationId))
        .findFirst()
        .orElseThrow(() -> new IllegalArgumentException("Registration not
found: " + registrationId));
}

private boolean isAlreadyRegistered(User attendee, Event event) {
    return registrations.stream().anyMatch(registration ->
registration.getAttendee().equals(attendee))

```

```

        &&
        registration.getEvent().getEventId().equalsIgnoreCase(event.getEventId())
    );
}
}

```

11) SessionService.java

```

package com.smarteventplatform.service;

import com.smarteventplatform.model.Event;
import com.smarteventplatform.model.Session;
import com.smarteventplatform.model.Venue;

import java.util.List;

public class SessionService {
    private int nextId = 1;

    public Session createSession(String title, String speaker, String timeSlot, Venue
venue, int capacity) {
        String sessionId = String.format("S%03d", nextId++);
        return new Session(sessionId, title, speaker, timeSlot, venue, capacity);
    }

    public List<Session> searchBySpeaker(List<Event> events, String speakerKeyword) {
        if (events == null || speakerKeyword == null || speakerKeyword.isBlank()) {
            return List.of();
        }

        String normalizedKeyword = speakerKeyword.toLowerCase();
        return events.stream()
            .flatMap(event -> event.getSessions().stream())
            .filter(session ->
session.getSpeaker().toLowerCase().contains(normalizedKeyword))
            .toList();
    }

    public List<Session> searchByTimeSlot(List<Event> events, String timeSlot) {
        if (events == null || timeSlot == null || timeSlot.isBlank()) {
            return List.of();
        }

        return events.stream()
            .flatMap(event -> event.getSessions().stream())
            .filter(session -> session.getTimeSlot().equalsIgnoreCase(timeSlot))
            .toList();
    }
}

```

12) UserService.java

```
package com.smarteventplatform.service;

import com.smarteventplatform.model.User;
import com.smarteventplatform.model.UserRole;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class UserService {
    private final List<User> users = new ArrayList<>();
    private int nextId = 1;

    public User registerUser(String name, String email, String password, UserRole
role) {
        if (isEmailRegistered(email)) {
            throw new IllegalArgumentException("Email already registered: " + email);
        }

        User user = new User(nextId++, name, email, password, role);
        users.add(user);
        return user;
    }

    public User login(String email, String password) {
        return users.stream()
            .filter(user -> user.getEmail().equalsIgnoreCase(email) &&
user.getPassword().equals(password))
            .findFirst()
            .orElseThrow(() -> new IllegalArgumentException("Invalid email or
password."));
    }

    public List<User> getAllUsers() {
        return new ArrayList<>(users);
    }

    public User findById(int userId) {
        return users.stream()
            .filter(user -> user.getUserId() == userId)
            .findFirst()
            .orElseThrow(() -> new IllegalArgumentException("User not found: " +
userId));
    }

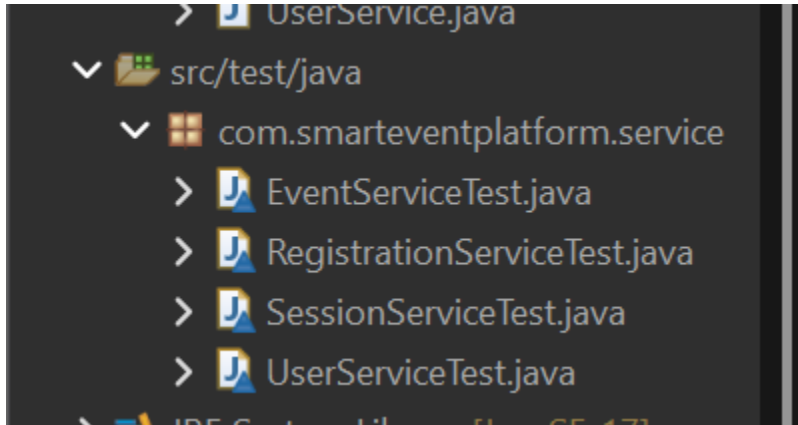
    private boolean isEmailRegistered(String email) {
        if (email == null) {
            return false;
        }
        Optional<User> existingUser = users.stream()
            .filter(user -> user.getEmail().equalsIgnoreCase(email))
            .findFirst();
        return existingUser.isPresent();
    }
}
```

```
}  
}
```

Output

```
****  
Problems Javadoc Declaration Console X  
<terminated> Main [Java Application] C:\Users\muham\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20210914-1920\jre\bin\java.exe  
Welcome to our  
Smart Event Management & Networking Platform  
  
--- Registered Users ---  
[User #1] Admin khan | admin@events.com | Role: ADMIN  
[User #2] Ammar Kabeer | ammar@events.com | Role: ORGANIZER  
[User #3] Usama | usama@gmail.com | Role: ATTENDEE  
[User #4] Ammar | ammar@gmail.com | Role: ATTENDEE  
  
Logged in as: Ammar Kabeer  
  
--- Event Details ---  
[Event E001] "Tech Conference 2026" on 2026-05-10 | Organizer: Ammar Kabeer | Sessions: 2  
[Session S001] "AI Workshop" by Dr. Sara @ 10:00 AM | Venue: Ghazkhan Hall | Capacity: 50  
[Session S002] "Cloud Computing" by Mr. Bilal @ 02:00 PM | Venue: Ghazkhan Hall | Capacity: 40  
  
--- Registrations ---  
[Reg R001] Usama -> "Tech Conference 2026" | Ticket: VIP | Status: PAID | QR: QR-R001-E001  
[Reg R002] Ammar -> "Tech Conference 2026" | Ticket: STANDARD | Status: FAILED | QR: N/A  
  
--- Updated Event ---  
[Event E001] "Tech Conference 2026 Updated" on 2026-05-10 | Organizer: Ammar Kabeer | Sessions: 2  
  
--- Payment Summary ---  
Paid registrations: 1  
  
--- Application Status ---  
Application execution completed.  
Thank you for using Smart Event Management & Networking Platform.
```

Testing classes



1.EventServiceTest.java

```
package com.smarteventplatform.service;

import com.smarteventplatform.model.Event;
import com.smarteventplatform.model.User;
import com.smarteventplatform.model.UserRole;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class EventServiceTest {

    @Test
    void organizerShouldCreateAndUpdateEvent() {
        UserService userService = new UserService();
        EventService eventService = new EventService();
        User organizer = userService.registerUser("Organizer", "org@test.com",
"pass123", UserRole.ORGANIZER);

        Event event = eventService.createEvent("Tech Expo", "2026-06-01", organizer);
        eventService.updateEvent(event.getId(), "Tech Expo Updated", null,
organizer);

        assertEquals("Tech Expo Updated", event.getTitle());
    }

    @Test
    void attendeeShouldNotCreateEvent() {
        UserService userService = new UserService();
        EventService eventService = new EventService();
    }
}
```

```

        User attendee = userService.registerUser("Attendee", "att@test.com",
"pass123", UserRole.ATTENDEE);

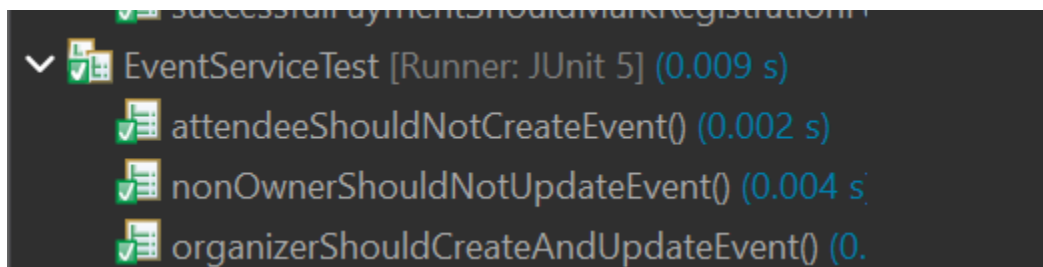
        assertThrows(IllegalArgumentException.class,
            () -> eventService.createEvent("Wrong Event", "2026-06-02",
attendee));
    }

    @Test
    void nonOwnerShouldNotUpdateEvent() {
        UserService userService = new UserService();
        EventService eventService = new EventService();
        User owner = userService.registerUser("Owner", "owner@test.com", "pass123",
UserRole.ORGANIZER);
        User otherOrganizer = userService.registerUser("Other", "other@test.com",
"pass123", UserRole.ORGANIZER);
        Event event = eventService.createEvent("Private Event", "2026-06-01", owner);

        assertThrows(SecurityException.class,
            () -> eventService.updateEvent(event.getEventId(), "Hacked Title",
null, otherOrganizer));
    }
}

```

1.Test Unit 1 Result:



2.RegistrationServiceTest.java

```

package com.smarteventplatform.service;

import com.smarteventplatform.model.Event;
import com.smarteventplatform.model.PaymentStatus;
import com.smarteventplatform.model.Registration;

```

```

import com.smarteventplatform.model.TicketType;
import com.smarteventplatform.model.User;
import com.smarteventplatform.model.UserRole;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertNotNull;
import static org.junit.jupiter.api.Assertions.assertNull;
import static org.junit.jupiter.api.Assertions.assertThrows;

class RegistrationServiceTest {
    @Test
    void successfulPaymentShouldMarkRegistrationPaidAndGenerateQrCode() {
        TestData testData = createTestData();

        Registration registration =
testData.registrationService.register(testData.attendee, testData.event,
TicketType.VIP);
        testData.registrationService.processPayment(registration, true);

        assertEquals(PaymentStatus.PAID, registration.getPaymentStatus());
        assertNotNull(registration.getQrCode());
    }

    @Test
    void failedPaymentShouldMarkRegistrationFailedAndNoQrCode() {
        TestData testData = createTestData();

        Registration registration =
testData.registrationService.register(testData.attendee, testData.event,
TicketType.STANDARD);
        testData.registrationService.processPayment(registration, false);

        assertEquals(PaymentStatus.FAILED, registration.getPaymentStatus());
        assertNull(registration.getQrCode());
    }

    @Test
    void attendeeCannotRegisterTwiceForSameEvent() {
        TestData testData = createTestData();
        testData.registrationService.register(testData.attendee, testData.event,
TicketType.VIP);

        assertThrows(IllegalStateException.class,
            () -> testData.registrationService.register(testData.attendee,
testData.event, TicketType.STANDARD));
    }

    private TestData createTestData() {
        UserService userService = new UserService();
        EventService eventService = new EventService();
        RegistrationService registrationService = new RegistrationService();

        User organizer = userService.registerUser("Organizer", "org@test.com",
"pass123", UserRole.ORGANIZER);
    }

```

```

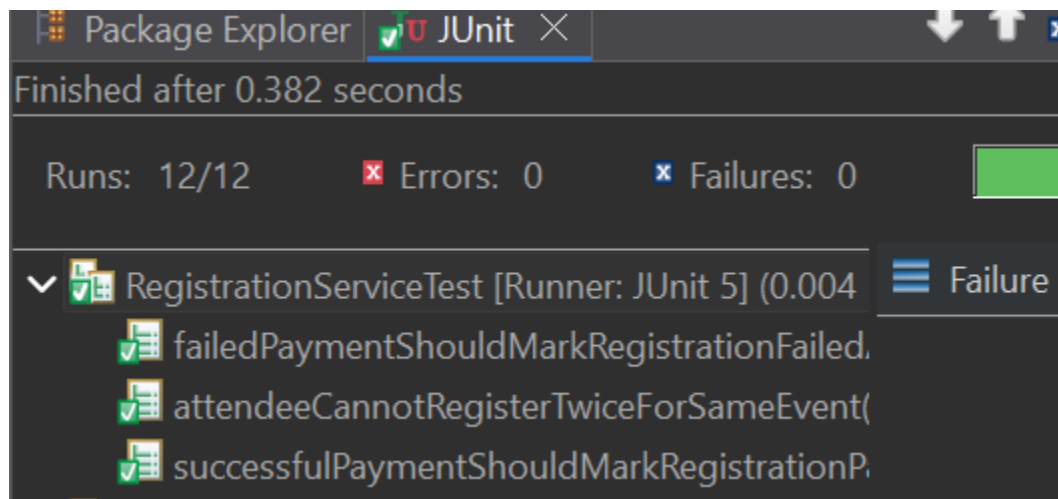
        User attendee = userService.registerUser("Attendee", "att@test.com",
"pass123", UserRole.ATTENDEE);
        Event event = eventService.createEvent("Cloud Day", "2026-08-01", organizer);

        return new TestData(registrationService, attendee, event);
    }

    private record TestData(RegistrationService registrationService, User attendee,
Event event) {
    }
}

```

2.Test unit 2 Result



3.SessionServiceTest.java

```

package com.smarteventplatform.service;

import com.smarteventplatform.model.Event;
import com.smarteventplatform.model.Session;
import com.smarteventplatform.model.User;
import com.smarteventplatform.model.UserRole;
import com.smarteventplatform.model.Venue;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.junit.jupiter.api.Assertions.assertTrue;

class SessionServiceTest {
    @Test
    void createSessionShouldGenerateSessionId() {
        SessionService sessionService = new SessionService();
        Venue venue = new Venue("V01", "Main Hall", 100);
    }
}

```



```

        Session session = sessionService.createSession("Java Basics", "Sara", "10:00 AM", venue, 50);

        assertTrue(session.getSessionId().startsWith("S"));
    }

    @Test
    void eventShouldRejectSameVenueAndSameTimeSlotConflict() {
        UserService userService = new UserService();
        EventService eventService = new EventService();
        SessionService sessionService = new SessionService();
        User organizer = userService.registerUser("Organizer", "org@test.com", "pass123", UserRole.ORGANIZER);
        Event event = eventService.createEvent("Java Day", "2026-07-01", organizer);
        Venue venue = new Venue("V01", "Main Hall", 100);

        Session firstSession = sessionService.createSession("Java Basics", "Sara", "10:00 AM", venue, 50);
        Session secondSession = sessionService.createSession("Java Advanced", "Bilal", "10:00 AM", venue, 40);
        event.addSession(firstSession);

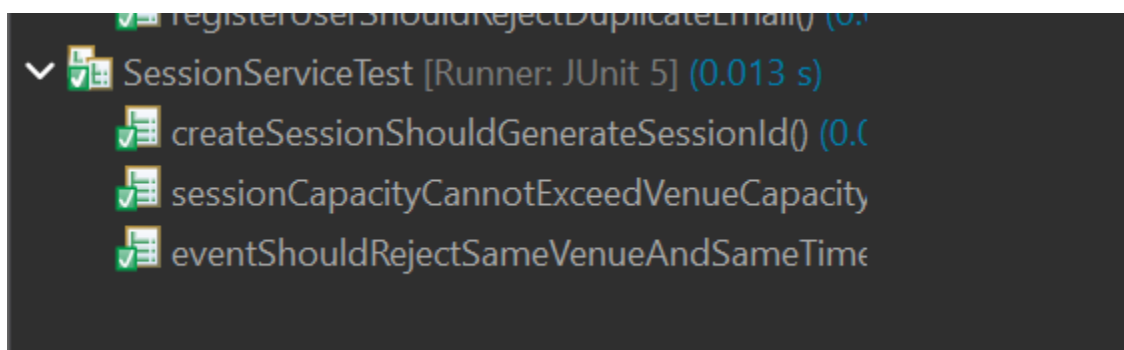
        assertThrows(IllegalStateException.class, () -> event.addSession(secondSession));
        assertEquals(1, event.getSessions().size());
    }

    @Test
    void sessionCapacityCannotExceedVenueCapacity() {
        SessionService sessionService = new SessionService();
        Venue venue = new Venue("V01", "Small Hall", 50);

        assertThrows(IllegalArgumentException.class, () -> sessionService.createSession("Big Talk", "Sara", "11:00 AM", venue, 100));
    }
}

```

3.Test unit 3 Result:



4.UserServiceTest.java

```
package com.smarteventplatform.service;

import com.smarteventplatform.model.User;
import com.smarteventplatform.model.UserRole;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class UserServiceTest {

    @Test
    void registerUserShouldSaveUserAndAllowLogin() {
        UserService userService = new UserService();

        User registeredUser = userService.registerUser("Usama", "usama@test.com",
"pass123", UserRole.ATTENDEE);
        User loggedInUser = userService.login("usama@test.com", "pass123");

        assertEquals("Usama", registeredUser.getName());
        assertEquals(registeredUser.getUserId(), loggedInUser.getUserId());
    }

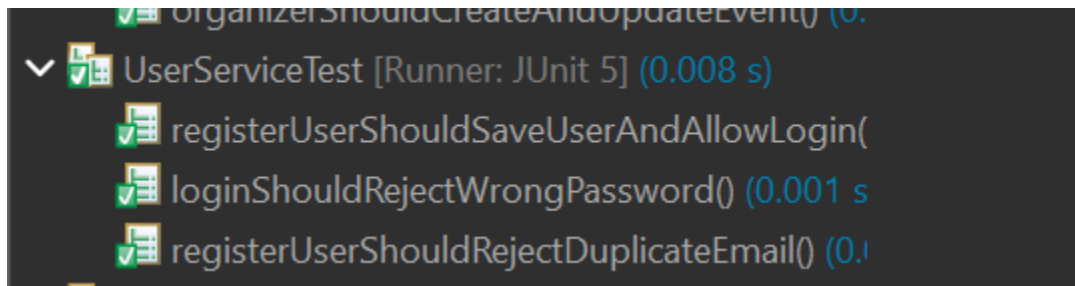
    @Test
    void registerUserShouldRejectDuplicateEmail() {
        UserService userService = new UserService();
        userService.registerUser("Usama", "usama@test.com", "pass123",
UserRole.ATTENDEE);

        assertThrows(IllegalArgumentException.class,
            () -> userService.registerUser("Ali", "usama@test.com", "pass123",
UserRole.ATTENDEE));
    }

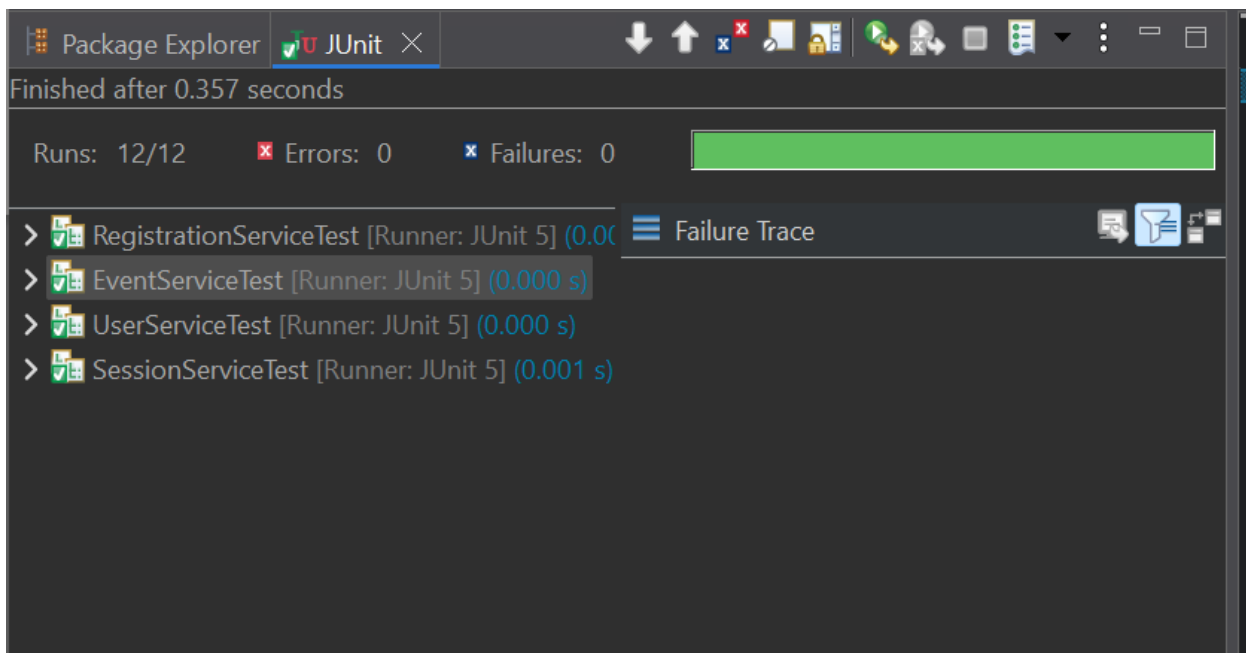
    @Test
    void loginShouldRejectWrongPassword() {
        UserService userService = new UserService();
        userService.registerUser("Usama", "usama@test.com", "pass123",
UserRole.ATTENDEE);

        assertThrows(IllegalArgumentException.class,
            () -> userService.login("usama@test.com", "wrong-password"));
    }
}
```

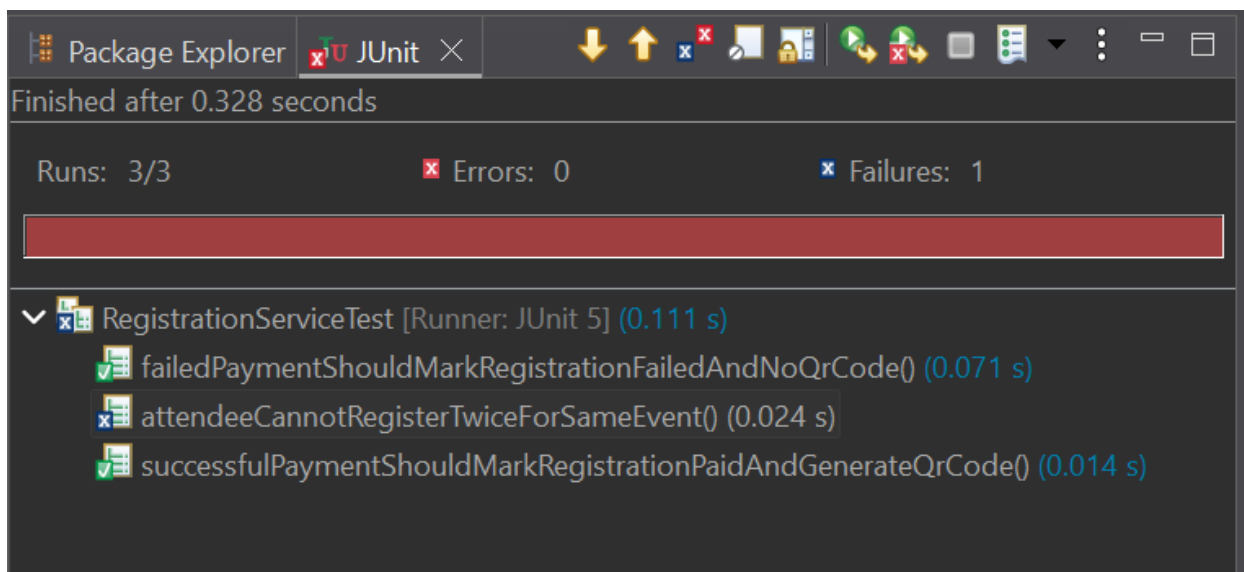
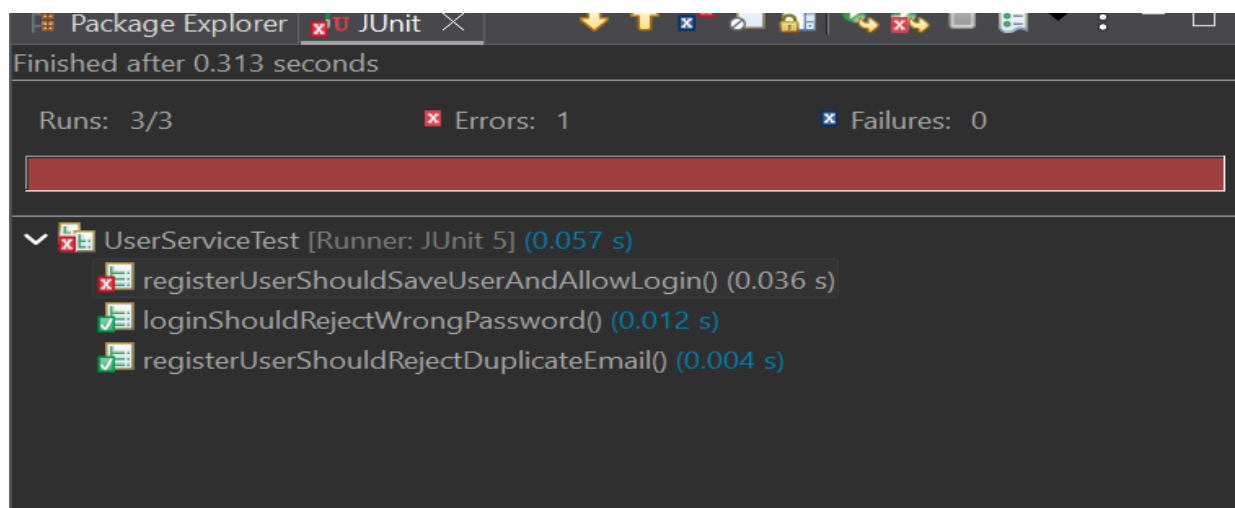
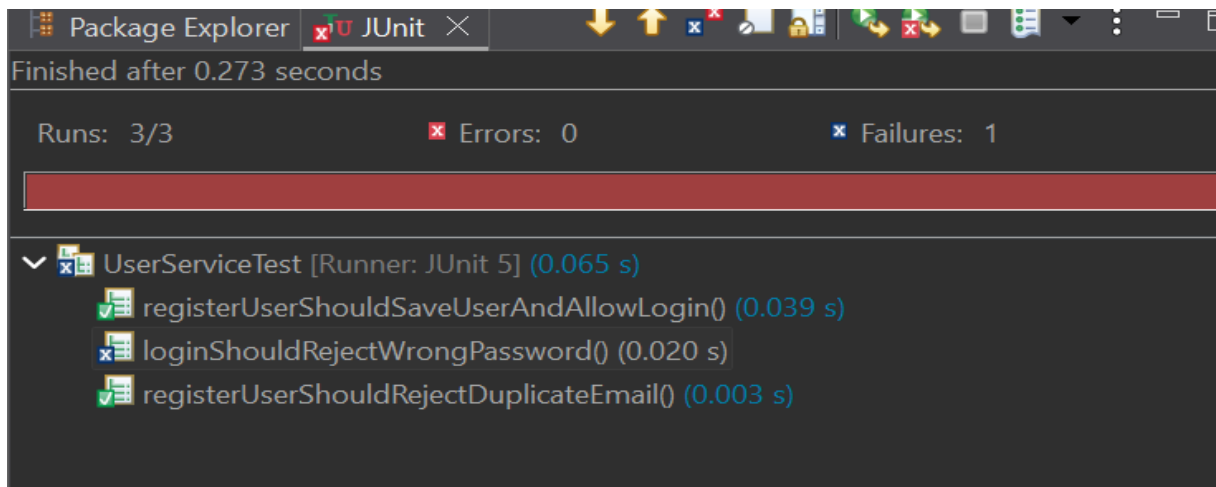
4.Test unit 4 Result



All units test Results:



FAILED TEST CASE TO VERIFY UNIT TESTING



Bug report table:

Test Case ID	Module	Test Scenario	Expected Result	Actual Result	Status
1	User Registration	Register user with valid details	User should register successfully	User registered successfully	Pass
2	Login System	Login with wrong password	System should reject login	Error message displayed	Pass
3	Event Creation	Organizer creates event	Event should be created	Event created successfully	Pass
4	Session Management	Add session with same venue and time	Conflict should be detected	Conflict exception generated	Pass
5	Registration	Same attendee registers twice	System should reject duplicate registration	Exception generated	Pass
6	Java Compatibility	Run project on Java 8	System should compile correctly	isBlank() error fixed using trim().isEmpty()	Fixed

Issues Encountered During Implementation and Testing

- Duplicate email registration caused validation errors.
- Invalid login credentials caused login failure.
- Passwords shorter than 6 characters were rejected.
- Duplicate event registration attempts generated exceptions.
- Java version compatibility issue occurred because isBlank() was not supported in Java 8. This was fixed using trim().isEmpty().
- Eclipse project folder opening issue occurred while importing the project and adding test classes. This was resolved by properly configuring the project structure and package folders inside Eclipse.